Grouped-Site LTA Cation Generator: Architecture Manual

Reverse-engineered and reconciled through package v7.15.0 (MS0-MS7 plus bounded sparse chloride occlus

2026-10-05

Contents

1. Purpose and status **3**

2. Package structure **4**

3. Core domain model **5**
- 3.1 Raw atom rows . . . 5
- 3.2 Candidate variants and physical sites . . . 5
- 3.3 Placed ions . . . 5
- 3.4 Population result . . . 6
- 3.5 Current symmetry representation . . . 6

4. Current end-to-end data flow **6**

5. Template expansion and physical-site grouping **6**
- 5.1 Symmetry expansion . . . 6
- 5.2 Ring-family classification . . . 6
- 5.3 Grouping positional variants . . . 7

6. Population constraints, chemistry, and volume **7**
- 6.1 Resolved allocations . . . 7
- 6.2 Capacity . . . 7
- 6.3 Formal charge neutrality . . . 8
- 6.4 Dry reference volume . . . 8

7. Current random population and unique batching **8**
- 7.1 Random population . . . 8
- 7.2 Grouped-site occupation state . . . 9
- 7.3 Current site-permutation construction . . . 9
- 7.4 Canonical orbit signature . . . 9

8. Density analysis and smoothing **9**
- 8.1 Ion-centered periodic density . . . 9
- 8.2 Clustering decision . . . 10
- 8.3 Scalar score . . . 10
- 8.4 Legacy random greedy relocation . . . 10
- 8.5 Symmetry-coupled smoothing . . . 10
- 8.6 User-facing modes . . . 11

9. Serialization and current invariants **11**

10. Maximum-symmetry population: exact problem statement **11**
- 10.1 Terminology . . . 11
- 10.2 Fixed-cell finite operation group . . . 11

10.3 Resolved occupation coloring . . . 12
10.4 Coordinate realization and stabilizer . . . 12
Occupation mode . . . 12
Exact static-coordinate mode . . . 13

11. Symmetry model and accepted parent group **13**
11.1 One immutable template model - implemented in MS0 . . . 13
11.2 Separate symmetry backend from chemical policy . . . 13
11.3 Parent group as an intersection . . . 13
11.4 Full group, variant-library policies, and actions . . . 14
Symmetry-completed library - default . . . 14
Strict supplied library - optional . . . 14
11.5 Symmetry data structure - implemented in MS0 . . . 15

12. Algebraic construction and numerical contracts **16**
12.1 Operation multiplication table . . . 16
12.2 Numerical meaning of exact symmetry . . . 16
12.3 Physical-site mapping independent of variant closure . . . 17
12.4 Variant-library construction and transition tables . . . 17
Symmetry-completed policy . . . 17
Strict policy . . . 17
12.5 Selected-orbit closure and composition . . . 18
12.6 Implemented subgroup infrastructure - MS1 . . . 18

13. Subgroup-orbit optimizer **19**
13.1 Orbit-coloring equivalence . . . 19
13.2 Exact orbit-coloring constraints . . . 19
13.3 Joint exact-mode feasibility . . . 19
13.4 Partial-transition site-stabilizer theorem . . . 20
13.5 Search order, conjugacy reduction, exhaustive certification, and budgets . . . 20
13.6 Feasibility solver - occupation layer implemented in MS2 . . . 21
13.7 Correctness and completeness . . . 21
13.8 Actual stabilizer verification . . . 22
13.9 Implemented MS2 occupation optimizer . . . 22
13.10 Implemented MS3 exact-coordinate optimizer . . . 22
13.11 Implemented MS6 exact-search acceleration . . . 23
Forced-variant lift . . . 23
Subgroup-orbit realization catalog . . . 23
Search and diagnostic accounting . . . 23

14. Occupation, realization, ranking, and batching **23**
14.1 Separate data models . . . 23
14.2 Primary objective and tie ordering . . . 24
Exact mode . . . 24
Occupation mode . . . 24
14.3 Bounded tie enumeration . . . 24
14.4 Symmetry-preserving smoothing - implemented in MS5 . . . 25

15. Module boundaries and public API **25**
15.1 Module boundaries . . . 25
15.2 Implemented CLI surface . . . 26
15.3 Implemented result types . . . 26

16. Staged implementation plan **27**
MS0 - template, parent group, and variant-library hardening - implemented in v7.5 . . . 27
MS1 - subgroup and orbit infrastructure - implemented in v7.6 . . . 27
MS2 - occupation optimizer - implemented in v7.7 . . . 27

MS3 - exact static-coordinate optimizer - implemented in v7.8 . . . 28
MS4 - integration, final scoring, reports, and serialization - implemented in v7.9 . . . 28
MS5 - symmetry-coupled smoothing - implemented in v7.10 . . . 28
MS6 - exact-search acceleration and scale validation - implemented in v7.11 . . . 28
MS7 - symmetry construction and search acceleration - implemented in v7.12 . . . 29

17. Verification strategy **29**
17.1 Algebraic group tests . . . 29
17.2 Orbit and optimizer tests . . . 29
17.3 Geometry and determinism tests . . . 30
17.4 End-to-end tests . . . 30

18. Current limitations and deferred scope after MS7 **31**

19. Algorithmic provenance and references **31**
19.1 Provenance boundary . . . 31
19.2 References . . . 32
v7.15 architecture delta: sparse chloride occlusion . . . 33

Current authority routing: supported Group-I/II framework-compensation species, exact integer populations, formal charges, and the +24 framework invariant are owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 charge-equivalent dry-volume model and empirical endpoint provenance are owned by `doc/specs/v7_14_group_i_ii_volume_reference_exten` the human-ratified sparse chloride/occlusion-counterion extension is owned by `doc/specs/v7_15_chloride_occlusion_e` This document imports those semantics and does not redefine them.

1. Purpose and status

This package generates Al24Si24O96 LTA initial structures from a symmetry-reduced CIF containing framework atoms and cation-template coordinates. It supports the qualified dry Group-I/II cation path and, in random mode only, the bounded v7.15 sparse chloride-occlusion initialization extension. Nearby split or off-center template coordinates are treated as positional variants of one physical ring site.

The current implementation provides:

- symmetry expansion of a CIF template into an explicit P1 framework;
- grouping of positional variants into physical 4-ring, 6-ring, and 8-ring sites;
- exact or ratio-based Group I/II cation population constraints;
- random population of physical sites;
- independent framework-compensation and sparse chloride/counterion charge validation in v7.15 random mode;
- deterministic alpha/beta cage-center chloride initialization with fail-closed template identity and collision checking;
- composition-dependent dry-LTA volume interpolation and isotropic rescaling using framework-compensating cations only;
- symmetry-inequivalent random batching on grouped-site occupations;
- maximum-order grouped-site occupation optimization with exact integer constraints and an optimality certificate;
- exact off-center variant realization with geometric coordinate-stabilizer and secondary occupation-stabilizer certificates;
- proof-preserving exact-search acceleration through forced-variant lifting and subgroup-orbit realization catalogs;
- detailed subgroup-operation and occupied-orbit diagnostics;
- periodic cation-density analysis and greedy de-clustering;
- CIF, VASP POSCAR, and LAMMPS data output.

The v7.14 line established the complete MS0-MS7 cation-symmetry path, which v7.15 preserves unchanged for non-occluded structures: immutable template loading, complete affine parent-group construction, variant-library

policies, integer group tables, exhaustive subgroup enumeration, subgroup orbits, exact occupation coloring, selected-orbit off-center realization, geometric coordinate stabilizers, maximum-order certificates, proof-preserving forced-variant and catalog acceleration, provenance-derived framework actions, generator-orbit variant completion, bit-mask subgroup search, hybrid direct occupation enumeration, chemistry/volume-aware structure assembly, serialization and read-back stabilizer verification, detailed subgroup diagnostics, and symmetry-coupled density smoothing over complete required-subgroup orbits.

2. Package structure

`lta_cation_population_grouped.py` - implemented. CIF parsing, template expansion, site grouping, role-aware random population, independent framework/occlusion charge checks, canonical sparse cage placement and collision validation, dry-volume checks, symmetry-unique dry batching, reports, writers, and the user-facing MS4 mode switch. Maximum-symmetry work is delegated to the dedicated MS0-MS7 modules.

`lta_cation_density_grouped.py` - implemented and optimized in MS7. Periodic cation-density metrics, clustering thresholds, reusable report scoring, CSV export, and optional density-grid output.

`generate_smooth_lta_cations_grouped.py` - implemented through MS5 plus v7.15 random occlusion. Legacy random regeneration/single-ion repair preserves cation-role metadata and finalizes chloride only after cation smoothing; the separate maximum-symmetry path uses complete-orbit smoothing and rejects occlusion before certification.

`lta_cation_template.py` - implemented in MS0. Immutable deterministic template model with expansion provenance, grouped-site anchors, fingerprints, and stable identifiers.

`lta_cation_symmetry.py` - implemented in MS0 and accelerated in MS7. Complete affine operation group, provenance-derived or vectorized framework actions, generator-orbit variant completion, integer tables, site/variant actions, and validation reports.

`lta_cation_subgroups.py` - implemented in MS1 and accelerated in MS7. Validated finite-group adapter, integer-bit-mask subgroup closure, exhaustive bounded subgroup enumeration, policy-dependent subgroup conjugacy, and subgroup orbit partitions.

`lta_cation_maxsym.py` - occupation and exact-coordinate layers implemented in MS2-MS3 and accelerated in MS6-MS7. Exact population resolution, small-space direct raw-occupation enumeration, subgroup-orbit coloring, selected-variant feasibility, partial-transition ambiguity handling, geometric occupation/coordinate stabilizers, canonical signatures, forced-variant lifting, subgroup-orbit realization catalogs, bounded ties, and optimality certificates.

`lta_cation_maxsym_io.py` - implemented in MS4 and extended in MS5-MS6. Converts certified solutions into `PopulationResult`, reuses chemistry and volume logic, applies post-symmetry canonical/density ranking and hard pair-distance filters, exposes a public fully verified member builder for MS5 trials, serializes through existing writers, verifies read-back composition, geometry, and full species-labeled stabilizers, and reports subgroup conjugacy, operation, and occupied-orbit diagnostics.

`lta_cation_symmetry_smoothing.py` - implemented in MS5. Complete-orbit variant reorientation, occupied-to-empty relocation, species exchange, deterministic density descent, required-stabilizer verification, bounded search, batch uniqueness checks, and smoothing reports.

`tests/test_unique_batch.py` - implemented. Focused regression tests for grouped-site symmetry equivalence and unique batching.

`tests/test_ms0_symmetry_model.py` - implemented in MS0. Algebraic, geometric, variant-policy, kernel, determinism, and spglib-focused tests.

`tests/test_ms1_subgroups.py` - implemented in MS1. Cyclic, dihedral, crystallographic-action, conjugacy-policy, determinism, and bounded-search tests.

`tests/test_ms2_occupation_optimizer.py` - implemented in MS2. Constraint resolution, exact orbit coloring, cyclic/dihedral/Klein-group optima, brute-force oracle comparison, canonicalization, tie bounding, and crystallographic integration.

tests/test_ms3_exact_coordinate_optimizer.py - implemented in MS3. Completed/strict selected-orbit realization, site-stabilizer rejection, coordinate/occupation stabilizer separation, ambiguity propagation, geometric canonicalization, deterministic tie handling, search guards, and crystallographic integration.

tests/test_ms4_integration_serialization.py - implemented in MS4. Chemistry/volume assembly, occupation realization, hard-filter semantics, reports, CLI integration, deterministic multi-output naming, tamper rejection, and CIF/POSCAR/LAMMPS round-trip stabilizer verification.

tests/test_ms5_symmetry_smoothing.py - implemented in MS5. Complete-orbit relocation, equivariant variant reorientation, species exchange, required-subgroup validation, bounded-search failure, batch/report integration, and smooth-CLI round-trip verification.

tests/test_ms6_exact_search_acceleration.py - implemented in MS6. Forced-variant lifting on the 24/16/6 LTA-capacity fixture, low-symmetry exact certification under a one-node variant budget, subgroup-catalog reuse, deterministic repetition, report diagnostics, and CIF round-trip verification.

tests/test_ms7_performance_optimizations.py - implemented in MS7. Accelerated construction diagnostics, total variant closure, direct/subgroup solver equivalence, bit-mask subgroup closure, density-report reuse, and the real 48-operation Na-LTA order-four regression.

pyproject.toml - added in MS0. Minimal packaging and pytest configuration; `pytest -q` works from the package root.

doc/specs/ - added in v7.1 and extended through MS7. Reverse-engineered contracts for the original executable modules plus implementation specifications for template/symmetry construction, subgroup infrastructure, occupation optimization, exact coordinate realization, exact-search acceleration, verified serialization, and symmetry-coupled smoothing.

doc/archs/ - added in v7.1 and revised through v7.12. System architecture, mathematical contracts, implemented stages, and verification plans.

3. Core domain model

3.1 Raw atom rows

`AtomRow` represents one CIF atom-site row after parsing:

```
AtomRow(label, element, frac, occ)
```

The minimal CIF reader retains only cell parameters, symmetry operations, atom labels, element symbols, fractional coordinates, and occupancy.

3.2 Candidate variants and physical sites

A raw template coordinate becomes a `CandidateVariant`:

```
CandidateVariant(frac, site_label, ring_class, site_id, variant_id)
```

Nearby variants in the same ring family are grouped into a `PhysicalSite`:

```
PhysicalSite(site_id, ring_class, site_label, variants[])
```

The physical occupancy constraint is one ion per `PhysicalSite`, not one ion per coordinate variant.

3.3 Placed ions

A selected ion is represented by:

```
PlacedIon(element, frac, site_label, ring_class, site_id, variant_id)
```

The grouped site controls occupation identity. `variant_id` records which alternative coordinate was selected.

3.4 Population result

`PopulationResult` is the complete generated state. It contains the final `pymatgen.Structure`, expanded framework atoms, grouped sites, placed ions, resolved allocations, chemical counts, volume metadata, grouping cutoffs, and random seed.

3.5 Current symmetry representation

`SiteSymmetryGroup` stores a finite permutation group acting on the ordered grouped sites:

$$\pi_g : \mathcal{P} \to \mathcal{P}.$$

Only the induced grouped-site permutations are retained. Distinct affine crystallographic operations that induce the same grouped-site permutation are collapsed. Off-center variants are not part of this representation.

This representation is sufficient for the current unique-batch definition. It is insufficient for exact atomic-coordinate symmetry and must be extended before maximum-symmetry population is implemented.

4. Current end-to-end data flow

```
input CIF
  -> minimal CIF parse
  -> symmetry expansion
  -> separate framework and template atoms
  -> classify template labels as 4R / 6R / 8R
  -> group nearby coordinate variants into physical sites
  -> resolve integer Group I/II cation allocations
  -> validate capacity and charge neutrality
  -> randomly choose physical sites and one variant per occupied site
  -> interpolate dry reference volume
  -> isotropically rescale lattice
  -> build explicit P1 structure
  -> optional density analysis and greedy repair
  -> optional grouped-site symmetry deduplication
  -> species-block normalization for serialization
  -> CIF / POSCAR / LAMMPS data output
```

5. Template expansion and physical-site grouping

5.1 Symmetry expansion

For every input atom row with fractional coordinate $\mathbf{f}$, each CIF operation g generates

$$\mathbf{f}' = g(\mathbf{f}) \bmod 1.$$

Expanded rows are deduplicated by element, label, and rounded fractional coordinate. If no CIF symmetry operation is present, identity is used.

5.2 Ring-family classification

Template labels are mapped to ring classes through `--site-map`. The default is

`NA1:6,NA2:8,NA3:4`

Unmapped template atoms are retained in `ignored_template_atoms` and are not populated.

5.3 Grouping positional variants

Within each ring family, raw candidate coordinates form an undirected proximity graph. Two candidates are connected when:

1. their minimum-image distance is no larger than the ring-family cutoff; and
2. their labels agree when `group_by_label=True`.

Connected components become physical sites. The implementation uses a disjoint-set forest with path compression. This data structure follows the standard set-union method analyzed by Tarjan [6]; the grouping criterion and its use for LTA split positions are package-specific.

The cutoff is a semantic parameter, not merely a numerical tolerance. A cutoff that is too small overcounts one physical site as several sites; a cutoff that is too large merges distinct ring sites.

6. Population constraints, chemistry, and volume

Let the supported cation alphabet be

$$\mathcal{C} = \{\mathrm{H}, \mathrm{Li}, \mathrm{Na}, \mathrm{K}, \mathrm{Rb}, \mathrm{Cs}, \mathrm{Fr}, \mathrm{Be}, \mathrm{Mg}, \mathrm{Ca}, \mathrm{Sr}, \mathrm{Ba}, \mathrm{Ra}\},$$

with formal charge $z_i = 1$ for Group I and $z_i = 2$ for Group II. In code this ordering is `SUPPORTED_CATIONS`; occupation-state ordering is `empty` followed by this sequence.

6.1 Resolved allocations

For each ring family $r \in \{4, 6, 8\}$ and supported Group I/II cation species α, the resolved allocation is an integer

$$N_{r,\alpha} \in \mathbb{Z}_{\geq 0}.$$

Ratio inputs are independently rounded within each ring family:

$$N_{r,\alpha} = \mathrm{round}(x_{r,\alpha}|\mathcal{P}_r|).$$

Counts and ratios cannot be supplied simultaneously for the same family.

6.2 Capacity

Each physical site can hold at most one ion:

$$\sum_{\alpha} N_{r,\alpha} \leq |\mathcal{P}_r|.$$

Optional auto-rebalancing moves overflow ions to another ring family according to fixed species preferences:

```
Li: 4R > 6R > 8R
Na: 6R > 8R > 4R
K : 8R > 6R > 4R
```

The remaining supported cations use the deterministic fallback order `4R > 6R > 8R`; no equilibrium or energetic preference is implied for that fallback.

This preserves global species counts but changes the user's requested ring-family counts.

6.3 Formal charge neutrality

The supported production model is a dry, fully condensed **Al24Si24O96** LTA framework (Si:Al = 1:1) compensated by the supported Group I (+1) and Group II (+2) cations. Its exact charge invariant is

$$\sum_i z_i N_i = 24.$$

The user-facing generation paths reject a framework outside Al24Si24O96. Non-24-Al LTA cells, other Si:Al ratios, and alternative cell normalizations are deferred to a future scientific/numerical extension. The code also rejects residual supported cations in the framework subset, non-unit framework occupancy, unsupported framework elements, and non-neutral cation totals.

6.4 Dry reference volume

For the supported Al24Si24O96 reference cell, the empirical corner volumes $V_i^{(24)}$ are taken from or derived from the cation-specific crystallographic sources [1,2,14-17]. Group I/II mixtures use charge-equivalent weights:

$$w_i = \frac{z_i N_i}{24}, \qquad \sum_i w_i = 1,$$

$$V_{\mathrm{ref}} = \sum_i w_i V_i^{(24)} = \frac{1}{24} \sum_i z_i N_i V_i^{(24)}.$$

Li, K, Rb, Cs, Mg, and Ba effective corners are affine extrapolations where the bound source is a mixed Na/M composition. Ca is a direct fully exchanged/dehydrated endpoint. Sr uses a **QUESTIONABLE effective corner** anchored to the two Pluth-Smith dehydrated Fm-3c refinements, which report `a=24.674(1)` and `24.687(1) A`. The package preserves its established `24.68 A` corner by taking their arithmetic midpoint (`24.6805 A`) and rounding to `0.01 A`; this is an explicit package modeling convention, not a claim that the paper reports one exact `24.68 A` pure-Sr cell. Pluth and Smith also report approximately 5.7 Sr per 12-Al pseudocell and an extra-framework Al complex, so the corner is not represented as a clean ideal Sr6-A endmember. H, Be, Fr, and Ra use explicit Na-A placeholders because v7.14 binds no species-specific dry-LTA volume reference for those species; this is not an exhaustive literature-absence claim.

The fixed denominator 24 is part of the present scientific model. It must not be generalized to `/N_Al` without a future definition of reference-volume normalization and physical cell scaling.

The requested target is

$$V_{\mathrm{target}} = f_V V_{\mathrm{ref}},$$

where `--volume-factor` is f_V. The lattice is rescaled isotropically by

$$\lambda = \left(\frac{V_{\mathrm{target}}}{V_{\mathrm{input}}} \right)^{1/3}.$$

Fractional coordinates and lattice shape are preserved.

7. Current random population and unique batching

7.1 Random population

For each ring family and species, the code samples the requested number of currently available physical sites without replacement. One coordinate variant is then chosen uniformly from each occupied site.

The algorithm guarantees exact resolved counts and single occupancy. It does not represent equilibrium probabilities or energetic site preferences.

7.2 Grouped-site occupation state

The ordered physical-site set is

$$\mathcal{P} = \mathcal{P}_4 \sqcup \mathcal{P}_6 \sqcup \mathcal{P}_8.$$

A population is encoded as

$$s : \mathcal{P} \to \{\emptyset\} \cup \mathcal{C}.$$

Different off-center variants of the same physical site produce the same state.

7.3 Current site-permutation construction

For each grouped site, the code computes a periodic centroid by unwrapping variants relative to the first variant and averaging. Each input CIF operation transforms the centroid, which is matched to a unique centroid in the same ring family within `--symmetry-map-tolerance`.

Every accepted operation must induce a bijection. The resulting generator permutations are closed under composition by direct finite-group closure.

Current limitation: mapping by centroid can hide differences in variant geometry. The implementation also collapses affine operations that have the same grouped-site action.

7.4 Canonical orbit signature

For each induced permutation π_g, the occupation signature is transformed. The canonical representative is

$$C(s) = \min_{g \in G_P} \text{tuple}(\pi_g s),$$

where G_P is the closed grouped-site permutation group and the minimum is lexicographic.

Two random populations are considered equivalent exactly when their canonical signatures agree.

This use of crystal symmetry to reduce occupational configurations follows the general group-action principle used in derivative-structure enumeration and symmetry-adapted site-occupancy methods [3,4,9]. The present fixed-cell grouped-site implementation and its direct canonical tuple comparison are custom to this package.

8. Density analysis and smoothing

8.1 Ion-centered periodic density

For selected cations, the minimum-image distance is d_{ij}. The code defines the unnormalized local Gaussian score

$$\rho_i = \sum_{j \neq i} \exp\left(-\frac{d_{ij}^2}{2\sigma^2}\right).$$

It reports the mean, standard deviation, maximum, maximum-to-mean ratio, maximum z-score, and minimum pair distance. This is a package-specific clustering heuristic, not a normalized probability-density estimate.

8.2 Clustering decision

A structure is flagged when any enabled condition fails:

$$\frac{\rho_{\max}}{\bar{\rho}} > R_{\max}, \qquad \frac{\rho_{\max} - \bar{\rho}}{\sigma_\rho} > Z_{\max}, \qquad d_{\min} < d_{\text{threshold}}.$$

For zero or one selected cation, the density ratios are zero and the minimum pair distance is infinity.

8.3 Scalar score

The repair objective is

$$S = \frac{\rho_{\max}}{\bar{\rho}} + 1000 \max(0, d_{\text{threshold}} - d_{\min}).$$

The distance term is present only when a threshold is supplied.

8.4 Legacy random greedy relocation

The random repair loop selects the cation with the largest local density. It samples empty physical sites in the same ring family, evaluates every variant at each sampled destination, and accepts the trial with the lowest score if it improves the current structure.

This is an original local heuristic. It preserves species counts, ring-family counts, charge neutrality, and volume. It does not preserve a nontrivial subgroup symmetry.

8.5 Symmetry-coupled smoothing

For a certified member, choose a required subgroup

$$H_{\text{req}} \leq H^{(0)}_{\text{coord}}.$$

The default is the complete initial coordinate stabilizer. The physical sites are partitioned into H_{req}-orbits. MS5 enumerates three deterministic local move families:

1. replace the selected variants on one occupied orbit by another exact equivariant realization;
2. move one species from a complete occupied orbit to an equal-size empty orbit in the same ring family;
3. exchange species labels between equal-size occupied orbits in the same ring family.

Every trial is rebuilt through the MS4 chemistry/volume layer. Its full coordinate stabilizer is recomputed, and the trial is accepted only if

$$H_{\text{req}} \leq H^{\text{trial}}_{\text{coord}}.$$

The best improving trial is selected by

$$(S, -d_{\min}, C_{\text{coord}}, \text{move metadata}).$$

The search stops when the density criteria pass, no improving certified move exists, or a move limit is reached. It guarantees subgroup preservation, not a global density optimum.

`min_pair_distance` is a soft repair target with a score penalty. `hard_min_pair_distance`, when enabled, rejects every trial below the stated distance. Incomplete candidate or variant searches raise explicit errors and do not emit a false local-optimum claim.

8.6 User-facing modes

- random `regenerate`: reject a clustered draw and generate another;
- random `move`: generate once and apply single-ion relocation;
- random `move-then-regenerate`: repair each draw, then regenerate if needed;
- `maximum-symmetry`: generate a certified MS4 member, apply MS5 orbit-coupled smoothing, serialize, and verify the final stabilizer after read-back.

For a legacy random unique batch, grouped-site symmetry deduplication is applied after repair. For a maximum-symmetry batch, MS5 preserves the existing uniqueness policy and raises if independently repaired members collapse to one canonical signature.

9. Serialization and current invariants

Before writing, a copy of the structure is stably grouped by element in the order

`Si Al O H Li Na K Rb Cs Fr Be Mg Ca Sr Ba Ra`

This avoids repeated POSCAR species blocks and makes CIF and LAMMPS output easier to inspect. The in-memory physical-site ordering is unchanged.

Supported output names are:

```
*.cif
*.POSCAR or POSCAR
*.vasp
*.data
```

LAMMPS data uses `atom_style atomic`; no per-atom charge column is written.

Current hard invariants are:

1. one ion at most per physical site;
2. exact resolved species counts per ring family unless explicit auto-rebalancing is enabled;
3. formal cation charge satisfies $\sum_i z_i N_i = 24$ for the supported Al24Si24O96 framework;
4. final volume equal to the requested target within floating-point precision;
5. deterministic results for identical inputs and seeds;
6. unique-batch members have distinct grouped-site canonical signatures;
7. output contains one contiguous atom block per element.

10. Maximum-symmetry population: exact problem statement

10.1 Terminology

This feature seeks a population with the **maximum preserved symmetry order**. The term *maximal subgroup* is reserved for a subgroup that is inclusion-maximal. A maximal subgroup need not have maximum order among all feasible stabilizers.

The implementation must therefore distinguish:

- **maximum-order feasible stabilizer**: a feasible stabilizer with largest cardinality;
- **inclusion-maximal feasible stabilizer**: a feasible stabilizer not contained in another feasible stabilizer;
- **maximum-symmetry structure**: an emitted structure whose verified stabilizer has maximum order under the selected realization mode.

10.2 Fixed-cell finite operation group

Let $\mathcal{G}$ be the parent space group accepted for the input structure, and let T_{cell} be the translation subgroup of the actual simulation cell. The finite operation group relevant to fixed-cell ordering is

$$G_{\text{cell}} = \mathcal{G}/T_{\text{cell}}.$$

The search keeps the input/output cell translation lattice fixed. It may remove centering or internal translations represented as nontrivial operations within that cell, but it does not search cell-enlarging ordered structures.

For the current primitive FCC LTA cell,

$$T_{\text{cell}} = T_{\text{primitive}}.$$

If a future input is conventional or a supercell, T_{cell} rather than the primitive translation subgroup remains authoritative.

10.3 Resolved occupation coloring

All user fractions must first be converted to exact integer counts. For each ring family $r \in \{4, 6, 8\}$ and state

$$\alpha \in \{\emptyset\} \cup \mathcal{C},$$

the optimizer receives

$$N_{r,\alpha} \in \mathbb{Z}_{\geq 0}.$$

A grouped-site occupation is a coloring

$$c : \mathcal{P} \to \{\emptyset\} \cup \mathcal{C}.$$

Its occupation stabilizer is

$$H_{\text{occ}}(c) = \{g \in G_{\text{site}} : c(gs) = c(s)\ \forall s \in \mathcal{P}\},$$

where G_{site} is the accepted parent group that preserves both the chemical framework and the grouped-site model.

10.4 Coordinate realization and stabilizer

Let X be the final species-labeled cation coordinates obtained by choosing one allowed positional variant at each occupied physical site. The coordinate stabilizer is

$$H_{\text{coord}}(X) = \{g \in G_{\text{site}} : gX = X\},$$

where equality preserves chemical species and is evaluated under the declared tolerance policy.

Necessarily,

$$H_{\text{coord}}(X) \leq H_{\text{occ}}(c).$$

The two supported optimization objectives are therefore distinct:

Occupation mode

$$\max_c |H_{\text{occ}}(c)|.$$

Variant selection is secondary and may lower the final coordinate symmetry. The report must state both stabilizers.

Exact static-coordinate mode

$$\max_X |H_{\text{coord}}(X)|.$$

Occupation feasibility and variant realizability must be tested jointly during subgroup search. It is incorrect to maximize $|H_{\text{occ}}|$ first and only afterward discover that its variants realize less coordinate symmetry than another occupation.

11. Symmetry model and accepted parent group

11.1 One immutable template model - implemented in MS0

Template parsing, symmetry provenance, framework atoms, and grouped candidate sites should be loaded once:

```
LTATemplateModel
  lattice
  framework_atoms
  physical_sites_by_ring
  ignored_template_atoms
  source_symmetry_operations
  grouping_parameters
  parser_provenance
```

This avoids the current duplication in which population and unique-batch symmetry logic independently read parts of the same CIF.

11.2 Separate symmetry backend from chemical policy

Two independent choices define the parent-operation candidates:

```
symmetry_backend: declared | spglib | auto
framework_equivalence: chemical | topological
```

Initial implementation scope should be limited to:

- `declared + chemical`: parse declared CIF operations and validate them against the actual species-labeled framework;
- `spglib + chemical`: detect operations from the expanded species-labeled framework;
- `auto + chemical`: validate declared operations and use spglib only when declared symmetry is absent or identity-only.

`topological` equivalence, in which selected framework species such as Si and Al are treated as equivalent, is a future pseudo-symmetry mode. It must never be reported as the physical symmetry of the chemically decorated structure.

Spglib detects operations numerically from lattice vectors, coordinates, and atomic types, subject to user-supplied tolerances [8,12]. It must be used for detection and validation only; the generator must not silently idealize framework coordinates.

11.3 Parent group as an intersection

The accepted parent group is not the lattice metric group alone. It is

$$G_{\text{site}} = \{g \in G_{\text{framework}} : g(\mathcal{P}_r) = \mathcal{P}_r \quad \forall r \in \{4, 6, 8\}\}.$$

An accepted operation must preserve:

1. the fixed lattice;
2. framework coordinates within tolerance;

3. unit framework occupancies and no coincident unlike species;
4. framework chemical species;
5. the complete grouped physical-site set;
6. 4R, 6R, and 8R family identity.

`site_label` is provenance metadata, not a symmetry color. Operations may map sites originating from differently named CIF rows when their grouped geometry and ring family agree.

The supplied off-center variant library does **not** define or filter G_{site}. A parent operation remains valid when it preserves the framework and physical-site model even if the supplied variant rows are incomplete or asymmetrically sampled.

This separation is required for correctness. After G_{site} is fixed, the variant layer either constructs a symmetry-completed active library or retains the supplied library strictly. Input-library closure is therefore not a prerequisite for parent-group construction; it is a policy-dependent coordinate-domain property.

11.4 Full group, variant-library policies, and actions

Let $G = G_{\text{site}}$ denote the complete finite affine-operation group retained after framework and physical-site validation. It has total actions

$$\phi_P : G \to \mathrm{Sym}(\mathcal{P})$$

on grouped physical sites and

$$\phi_F : G \to \mathrm{Sym}(\mathcal{F})$$

on framework atoms.

Each physical site s owns a finite variant fiber V_s. The framework/site parent group is constructed independently of the off-center variant library. The active coordinate domain is then selected by one of two explicit policies.

Symmetry-completed library - default

Starting from the user-supplied variants $\mathcal{V}_{\text{input}}$, construct

$$\widehat{\mathcal{V}} = \left\{ g(v) : g \in G,\ v \in \mathcal{V}_{\text{input}} \right\},$$

placing $g(v)$ in the already determined destination fiber $V_{g(s)}$ and periodically deduplicating within the variant tolerance. Because G is finite and closed, one full group sweep followed by validation produces a G-invariant active library. Every generated variant is an exact parent-symmetry image of a supplied coordinate; this policy does not introduce centroids, idealized centers, or arbitrary new positions.

The completed library must carry a validated total action

$$\phi_V : G \to \mathrm{Sym}(\widehat{\mathcal{V}}).$$

If periodic deduplication or tolerance ambiguity prevents a unique composition-consistent action, model construction fails with `VariantLibraryConstructionError`. It must not continue with an unverified completion.

Strict supplied library - optional

The strict policy uses only $\mathcal{V}_{\text{input}}$. Its geometric transformation is represented by a partial transition

$$A_g(s, v) = \begin{cases} (g(s), w), & g(v) \text{ matches exactly one } w \in V_{g(s)}, \\ \bot, & \text{no unique supplied match exists}, \end{cases}$$

for $v \in V_s$. The value $\perp$ records either a missing match or a tolerance ambiguity; the validation report distinguishes the two cases.

Define the strict-library-preserving group

$$G_{\mathcal{V}} = \left\{ g \in G : g(\mathcal{V}_{\text{input}}) = \mathcal{V}_{\text{input}} \right\}.$$

$G_{\mathcal{V}}$ must be validated as a subgroup of G. It is the largest parent subgroup whose conjugation preserves the strict exact-coordinate search domain.

The physical-site action may have a nontrivial kernel

$$K_P = \ker \phi_P.$$

Operations in K_P fix every grouped physical site but may move or permute off-center variants. The current v7 implementation collapses such operations; the new model must retain them.

The report should include

$$|G|, \qquad |\phi_P(G)|, \qquad |K_P|, \qquad |G_{\mathcal{V}}|,$$

plus input, generated, and active variant counts; variant-transition coverage; ambiguity diagnostics; and generation provenance.

This multiple-action formulation and the conjugacy properties of invariant domains follow standard finite group-action language [7]. Symmetry completion of the supplied LTA variant set, strict-domain reduction by $G_{\mathcal{V}}$, and their use in this optimizer are package-specific constructions.

11.5 Symmetry data structure - implemented in MS0

```
SymmetryModel
  operations: tuple[AffineSymmetryOperation, ...]
  multiplication_table: int[n_operations, n_operations]
  inverse_table: int[n_operations]
  identity_index: int
  physical_site_action: int[n_operations, n_sites]
  framework_action: int[n_operations, n_framework]
  variant_library_policy: symmetry_complete | strict
  input_variants: tuple[ActiveVariant, ...]
  active_variants: tuple[ActiveVariant, ...]
      # each ActiveVariant carries VariantProvenance records
  variant_transition: int[n_operations, n_active_variants]
      # destination global variant index; -1 when undefined in strict mode
  variant_transition_status: missing | unique | ambiguous
  variant_transition_residual: float[n_operations, n_active_variants]
  variant_owner_site: int[n_active_variants]
  library_preserving_group
  physical_site_image
  physical_site_kernel
  globally_closed_variant_action: bool
  tolerance_policy
  validation_report

VariantProvenance
  source: input | symmetry_generated
  source_variant_id
  generating_operation_id
```

```
  periodic_dedup_class
  mapping_residual

AffineSymmetryOperation
  rotation_frac: int[3,3]
  translation_frac: float[3]
  source_backend
  source_index
```

Distinct affine operations remain distinct even when their grouped-site permutations are identical. Under the symmetry-completed policy, variant transitions are total and validated. Under the strict policy, undefined transitions never remove an operation from G_{site}; they restrict only exact-coordinate feasibility.

12. Algebraic construction and numerical contracts

12.1 Operation multiplication table

Floating-point affine composition should be confined to symmetry-model construction. For fractional operations

$$g_i = (R_i, \mathbf{t}_i),$$

composition is

$$(R_i, \mathbf{t}_i)(R_j, \mathbf{t}_j) = \big(R_i R_j, R_i \mathbf{t}_j + \mathbf{t}_i\big) \pmod{\mathbb{Z}^3}.$$

After accepting the finite operation set, build

$$M_{ij} = k \quad \Longleftrightarrow \quad g_i g_j = g_k,$$

and an inverse table

$$I_i = j \quad \Longleftrightarrow \quad g_i g_j = e.$$

Subgroup closure, conjugation, and stabilizer calculations then operate only on integer operation indices. Subgroups should be represented as immutable bitsets or `frozenset[int]`.

Symmetry-model construction fails unless the operation table has:

- one identity;
- closure;
- one inverse per operation;
- physical-site and framework action consistency with multiplication;
- associativity, inherited and checked through the completed table on focused fixtures.

12.2 Numerical meaning of exact symmetry

"Exact" means exact relative to the accepted finite operation model and numerical tolerance policy. No floating-point crystal file can provide algebraic exactness without an idealized symbolic representation.

```
SymmetryTolerancePolicy
  framework_distance_ang
  site_distance_ang
  variant_distance_ang
  operation_translation_fractional
  angle_tolerance_deg
  postwrite_distance_ang
```

The validation report must record:

- symmetry backend and chemical-equivalence policy;
- every tolerance;
- maximum framework mapping residual;
- maximum grouped-site mapping residual;
- variant-library policy, generation and periodic-deduplication residuals, and transition-coverage statistics;
- post-serialization verification residual.

The post-write tolerance should not be looser than the construction tolerance unless output-format precision requires an explicit documented exception.

12.3 Physical-site mapping independent of variant closure

The physical-site permutation $\phi_P(g)$ must be constructed before and independently of off-center variant realizability.

The mapping hierarchy is:

1. use source-operation provenance from symmetry expansion when it provides an unambiguous physical-site image;
2. otherwise transform deterministic physical-site anchors and match sites by ring family, periodic geometry, and a local chemical-framework fingerprint;
3. solve the destination assignment one-to-one over the complete site family;
4. use the periodic centroid only as a candidate filter, never as the sole proof of equivalence;
5. use available variant matches as additional validation, not as a requirement that every source variant be present.

The local fingerprint should be invariant under coordinate wrapping and independent of CIF atom-row order. A practical first implementation may use sorted species-labeled distances from the site anchor to nearby framework atoms, with deterministic tie handling.

Physical-site mapping fails only when the site image is absent, non-bijective, or geometrically ambiguous. Missing off-center variants do not cause site-mapping failure.

12.4 Variant-library construction and transition tables

For each accepted operation g, source site s, and source variant $v \in V_s$, transform the coordinate and compare it only with variants in the already determined destination fiber $V_{g(s)}$.

Symmetry-completed policy

Apply every $g \in G$ to every input variant, insert the transformed coordinate into the destination fiber, and periodically deduplicate deterministically. Retain all provenance records even when several generated images collapse to one active variant.

After completion, build

```
variant_transition[g][global_variant_v]
    -> global_variant_w
```

and require a unique total action satisfying the group multiplication and inverse tables. Failure to construct a unique action is a model-construction error, not an indeterminate subgroup result.

Strict policy

Build

```
variant_transition[g][global_variant_v]
    -> global_variant_w | undefined
```

using only supplied variants. A transition is:

- **unique** when exactly one destination variant matches within tolerance;
- **missing** when none matches;
- **ambiguous** when more than one matches.

In both policies the hard invariant is

`destination_owner_site == physical_site_action[g][source_owner_site]`

Variant identifiers must follow a deterministic periodic-coordinate ordering, not incidental CIF row order.

Strict mode never chooses arbitrarily among ambiguous matches. It records ambiguity and propagates an **indeterminate** status through exact subgroup feasibility. Ambiguity does not reject the parent operation or occupation model, but an unresolved higher-order subgroup prevents an optimality certificate. A later extension may solve the small globally consistent matching problem explicitly.

The strict-library-preserving group $G_{\mathcal{V}}$ is obtained from operations whose strict transition rows are total bijections of the supplied active library. Its identity, closure, and inverses must be checked with the parent multiplication table.

12.5 Selected-orbit closure and composition

Exact mode requires an equivariant variant assignment only on variants selected for occupied subgroup orbits.

For subgroup $H \leq G_{\text{site}}$ and representative pair (s, v), define

$$\mathcal{O}_H(s, v) = \{A_h(s, v) : h \in H\}.$$

The representative is admissible only when:

1. $A_h(s, v) \neq \perp$ for every $h \in H$;
2. all generated pairs use active variants in their destination fibers;
3. whenever the compositions are evaluated on the generated set,

$$A_{h_1 h_2} = A_{h_1} \circ A_{h_2};$$

4. inverse transitions return every generated pair to itself;
5. no required transition is ambiguous.

Under the symmetry-completed policy, transition closure and composition are guaranteed globally after model validation, but the site-stabilizer fixed-point condition can still make a specific one-variant-per-site realization impossible. Under the strict policy, an incomplete global library can realize H whenever the selected orbit satisfies the conditions above.

Conjugacy reduction is valid only under a group that preserves the active coordinate domain. The completed policy is invariant under G; the strict policy is guaranteed invariant only under $G_{\mathcal{V}}$.

12.6 Implemented subgroup infrastructure - MS1

MS1 converts the validated MS0 operation tables into `FiniteGroupTables` and represents each subgroup as an immutable closed `frozenset[int]` with a deterministic generator witness. It implements:

- explicit subgroup closure and validation;
- exhaustive bounded subgroup-lattice enumeration by adjoining parent operations;
- deterministic descending-order subgroup traversal;
- conjugation gHg^{-1} and conjugacy orbits under a supplied admissible group C;
- subgroup orbit partitions for validated total actions;
- policy selection $C = G_{\text{site}}$ for occupation/completed exact mode and $C = G_{\mathcal{V}}$ for strict exact mode.

The enumeration search node is one distinct subgroup state. Exceeding the configured node guard raises `MaximumSymmetrySearchIncompleteError`; no partial lattice or optimality claim is returned. The implemented records are `Subgroup`, `SubgroupLattice`, `SubgroupConjugacyOrbit`, `SubgroupOrbitPartition`, and `SubgroupEnumerationReport`.

13. Subgroup-orbit optimizer

13.1 Orbit-coloring equivalence

For a candidate subgroup $H \leq G_{\text{site}}$, the physical sites split into orbits

$$\mathcal{P} = O_1 \sqcup \cdots \sqcup O_m.$$

An occupation preserves H if and only if it is constant on every H-orbit. This is the same group-subgroup orbit-splitting principle used in crystallographic Wyckoff-position splitting [5], applied here to custom grouped ring sites rather than necessarily literal Wyckoff positions.

13.2 Exact orbit-coloring constraints

For orbit O_k and state α, define binary variable $x_{k\alpha}$. Then

$$\sum_{\alpha} x_{k\alpha} = 1,$$

and for each ring family r and species α,

$$\sum_{O_k \subseteq \mathcal{P}_r} |O_k| x_{k\alpha} = N_{r,\alpha}.$$

Vacancy is an ordinary state. Its exact count is

$$N_{r,\emptyset} = |\mathcal{P}_r| - \sum_{\alpha \in \mathcal{C}} N_{r,\alpha}.$$

13.3 Joint exact-mode feasibility

Occupation mode declares H feasible when the orbit-coloring constraints have a solution.

Exact static-coordinate mode declares H feasible only when there exists an orbit coloring for which every occupied physical-site orbit has at least one representative variant whose selected H-orbit is closed under the active transition table. Variant feasibility is therefore part of subgroup feasibility, not a later best-effort conversion.

Under `symmetry-complete`, model construction first establishes a total variant action. Exact subgroup feasibility is then two-valued: feasible or infeasible.

Under `strict`, exact feasibility is three-valued during search:

- **feasible**: at least one complete coloring and selected-variant realization is certified;
- **infeasible**: every possible coloring and representative choice is disproved using only unique or missing transitions;
- **indeterminate**: no feasible realization is found, but at least one unresolved ambiguous transition could change the result.

Unused missing strict variants do not affect feasibility. A missing transition required by a selected orbit makes that representative infeasible. An ambiguous required transition makes that branch indeterminate, not infeasible. If any subgroup above a proposed optimum remains indeterminate, exact mode must raise `VariantTransitionAmbiguityError` and withhold the maximum-symmetry claim.

13.4 Partial-transition site-stabilizer theorem

Let s represent one occupied physical-site orbit under H, and let

$$H_s = \{h \in H : h(s) = s\}$$

be its site stabilizer. Assume the relevant partial transitions are unique and composition-consistent wherever they are defined.

An H-equivariant one-variant-per-site assignment generated from $v \in V_s$ exists if and only if

$$A_h(s, v) \neq \perp \qquad \forall h \in H,$$

and

$$A_k(s, v) = (s, v) \qquad \forall k \in H_s.$$

Proof. If an equivariant assignment exists, every operation in H must map the representative to an allowed selected variant, so all required transitions are defined. Every operation fixing s must also fix its selected variant, which gives the second condition. Conversely, define the variant at $h(s)$ by $A_h(s, v)$. If $h_1(s) = h_2(s)$, then $h_2^{-1}h_1 \in H_s$ fixes (s, v); composition consistency gives $A_{h_1}(s, v) = A_{h_2}(s, v)$. The assignment is therefore well defined and H-equivariant. □

This reduces exact realization to independent representative tests on occupied physical-site orbits, unless an additional inter-orbit physical constraint couples their choices. It does not require closure of unused variants.

13.5 Search order, conjugacy reduction, exhaustive certification, and budgets

Candidate subgroups are tested in descending order of subgroup order. The permitted conjugacy reduction depends on the realization mode and active variant-library policy.

Define the admissible conjugating group C by

$$C = \begin{cases} G_{\text{site}}, & \text{occupation mode,} \\ G_{\text{site}}, & \text{exact mode with symmetry-completed variants,} \\ G_{\mathcal{V}}, & \text{exact mode with strict variants.} \end{cases}$$

Subgroups $H_1, H_2 \leq G_{\text{site}}$ may share one feasibility test only when

$$H_2 = cH_1c^{-1} \qquad \text{for some } c \in C.$$

The active candidate domain is invariant under C, so feasibility is constant on these conjugacy orbits. In strict mode, $G_{\mathcal{V}}$ may be the identity; then every subgroup must be tested individually. It is incorrect to reduce strict exact-mode search by full G_{site} conjugacy unless the strict library is globally invariant.

Feasibility is monotone down the subgroup lattice:

- if H is feasible, every subgroup of H is feasible;
- if H is infeasible, every supergroup of H is infeasible.

The first feasible subgroup order proves the optimum for the selected realization mode only when all relevant subgroup conjugacy orbits of larger order have been exhausted. All relevant orbits of the optimum order must also be examined when every maximum-order symmetry type is requested.

Subgroups should be generated lazily and represented by immutable operation-index bitsets or `frozenset[int]`. The search records:

- variant-library policy and admissible conjugating-group order $|C|$;

- distinct subgroup states generated;
- conjugacy orbits tested under C;
- subgroup states pruned by known infeasibility;
- subgroup orders above the optimum that were completely exhausted;
- whether the optimum is proven.

A configurable search-node guard prevents pathological resource use. If the guard is reached before optimality is proven, the implementation raises `MaximumSymmetrySearchIncompleteError`. Likewise, unresolved ambiguity in any not-yet-excluded higher-order strict-mode subgroup raises `VariantTransitionAmbiguityError`. Neither condition may emit a structure labeled "maximum symmetry," and neither may silently return the best candidate found so far.

The default subgroup guard is

```
max_symmetry_search_nodes = 1_000_000
```

where one MS1 node is one distinct subgroup state discovered during exhaustive lattice construction. MS2 adds an independent

```
max_coloring_search_nodes = 1_000_000
```

where one coloring node is one distinct memoized dynamic-programming state $(i, \mathbf{n})$. Both counter definitions are stable and recorded in the certificate. Exceeding either guard returns no partial optimum. The expected primitive-LTA groups should remain far below these limits, but the correctness contract does not depend on that expectation.

13.6 Feasibility solver - occupation layer implemented in MS2

MS2 implements memoized integer dynamic programming over

$$(i, \mathbf{n}),$$

where i is the next subgroup orbit and $\mathbf{n}$ contains the remaining counts of vacancy and every supported cation for each ring family. The canonical state order is `empty` followed by `SUPPORTED_CATIONS` (`H, Li, Na, K, Rb, Cs, Fr, Be, Mg, Ca, Sr, Ba, Ra`).

The occupation solver applies:

- exact total-capacity equality;
- greatest-common-divisor compatibility;
- per-state subset-sum feasibility;
- remaining-capacity equality at every recursion depth;
- a shared distinct-DP-state guard that raises `OccupationColoringSearchIncompleteError` without returning a partial optimum.

The first feasible subgroup order is certified only after all larger full-parent conjugacy classes have been exhausted. Every candidate is canonicalized under the complete physical-site action, rescored by its actual occupation stabilizer, and checked with the orbit-stabilizer identity.

MS3 extends each feasible occupation coloring with selected-variant closure, site-stabilizer-fixed representatives, and feasible/infeasible/indeterminate exact-coordinate states. Independent occupied-site orbits are combined by a bounded deterministic Cartesian recursion. A brute-force solver is retained only as a small-fixture test oracle; no mixed-integer dependency is required.

13.7 Correctness and completeness

For every candidate occupation or coordinate realization X,

$$H_X = \mathrm{Stab}_{G_{\mathrm{site}}}(X)$$

is itself a subgroup of the parent group. The subgroup-first search includes H_X directly, or a representative in its conjugacy orbit under the admissible group C. Because C preserves the active candidate domain, conjugate representatives under C have the same feasibility status. Therefore,

$$\max_X |\operatorname{Stab}_{G_{\text{site}}}(X)| = \max_{\substack{H \le G_{\text{site}} \\ H \text{ feasible}}} |H|.$$

Testing policy-appropriate subgroup conjugacy orbits in descending order is globally complete when:

1. the feasibility definition matches the occupation or exact-coordinate objective;
2. the active domain is invariant under the chosen conjugating group C;
3. every required subgroup orbit of larger order is exhausted;
4. the search terminates with an explicit optimality certificate.

For occupation mode and symmetry-completed exact mode, $C = G_{\text{site}}$. For strict exact mode, $C = G_{\mathcal{V}}$; using the full parent group can skip a feasible subgroup and invalidate the certificate. Search-limit or unresolved-ambiguity exceptions are not maximum-symmetry results.

13.8 Actual stabilizer verification

A candidate constructed for subgroup H may accidentally preserve a supergroup. Every result must be rescored directly:

$$H_{\text{occ}} = \{g : g \cdot c = c\},$$

$$H_{\text{coord}} = \{g : g \cdot X = X\}.$$

The reported symmetry is the actual verified stabilizer, not the subgroup used to construct the candidate.

13.9 Implemented MS2 occupation optimizer

`lta_cation_maxsym.py` now provides:

- `ResolvedPopulationConstraints` and exact vacancy inference;
- `SiteOccupation` and `OrbitColoringSolution`;
- deterministic subgroup-orbit coloring with a bounded memoized search;
- canonical occupation signatures under the full parent action;
- actual occupation-stabilizer reconstruction and orbit-stabilizer validation;
- `first` and bounded `all` modes;
- `OccupationOptimalityCertificate` recording exhausted larger orders, subgroup and coloring search nodes, conjugacy coverage, tie truncation, and `optimality_proven`.

The stored occupation is the canonical parent-orbit representative. The construction subgroup is conjugated by the same canonicalizing operation and remains contained in the verified stabilizer. MS2 outputs abstract grouped-site occupations only; it does not select positional variants or write structures.

13.10 Implemented MS3 exact-coordinate optimizer

`lta_cation_maxsym.py` now also provides:

- `VariantAssignment`, `OrbitVariantRealization`, and `ExactVariantFeasibility`;
- completed-library total-action realization and strict partial-transition analysis;
- explicit site-stabilizer fixed-point and all-site equivariance checks;
- deterministic bounded combination of independent occupied-orbit choices;
- geometric species-labeled coordinate stabilizers evaluated from affine operations and selected fractional coordinates;
- full-parent geometric coordinate signatures, including strict images outside the active supplied domain;

- `first` and bounded `all` exact modes;
- `CoordinateOptimalityCertificate`, independent variant-search accounting, and ambiguity refusal.

The exact objective is lexicographic. MS3 first proves the maximum coordinate-stabilizer order. It then exhausts every subgroup class and occupation coloring at that coordinate order to maximize the occupation-stabilizer order. In `first` mode, one deterministic variant realization per feasible occupation is sufficient for this secondary proof; variant-level tie exhaustion is reserved for `all`.

Actual coordinate verification is geometric rather than ID-based. Once a complete assignment is known, operation g belongs to H_{coord} only when the transformed selected coordinate at every occupied source site matches the selected coordinate at the mapped destination site within `variant_distance_ang`. Transition ambiguity is therefore a search-state issue for unknown assignments, not an obstacle to verifying an already specified assignment.

MS3 returns in-memory exact solutions only. It does not call the existing writers or perform post-write symmetry verification; those are MS4 responsibilities.

13.11 Implemented MS6 exact-search acceleration

MS6 preserves the exact objective and adds two proof-producing accelerations.

Forced-variant lift

If every physical site has exactly one active variant and every parent operation maps the sole source variant uniquely to the sole destination variant, the coordinate assignment is forced. For every occupation c,

$$\mathrm{Stab}_G(X(c)) = \mathrm{Stab}_G(c).$$

The optimizer therefore delegates the search to MS2 and lifts the certified occupation result. It then recomputes both stabilizers geometrically and requires equality. This path consumes zero variant-search nodes and is recorded as `forced_variant_lift`.

Subgroup-orbit realization catalog

For a general multi-variant domain, realizability of one subgroup orbit depends on the subgroup, orbit, active variants, and transition relation, but not on which supported cation species labels the orbit. MS6 builds a `SubgroupVariantRealizationCatalog` once per tested subgroup and reuses it across all occupation colorings. Reuse consumes no additional representative-test budget.

The general strategy is recorded as `catalogued_subgroup_variant_search`. Ambiguous entries remain indeterminate and preserve the MS3 refusal semantics.

Search and diagnostic accounting

`CoordinateOptimalityCertificate` now records the exact-search strategy, fast-path status, catalog build/reuse counts, and that the primary coordinate order is certified before lower subgroup orders are skipped and optimum-order tie processing continues. MS4 reports the parent-conjugacy key, generators, finite operation orders, affine descriptors, and occupied coordinate-subgroup orbit sizes.

The detailed contract is in `doc/specs/lta_cation_exact_search_acceleration_spec.md`.

14. Occupation, realization, ranking, and batching

14.1 Separate data models

Occupation and coordinate realization are distinct decisions:

```
SiteOccupation
  state_by_site: tuple[empty | SUPPORTED_CATIONS, ...]
```

```
VariantAssignment
  variant_by_occupied_site: mapping[site_id, variant_id]

PlacedIon
  derived final coordinate record
```

This separation supports occupation canonicalization, exact variant feasibility, independent stabilizer calculations, and tests in which $H_{\mathrm{occ}} > H_{\mathrm{coord}}$.

14.2 Primary objective and tie ordering

The symmetry claim certified in MS2-MS3 and accelerated in MS6 is the primary maximum-order objective:

Exact mode

1. maximize $|H_{\mathrm{coord}}|$;
2. maximize $|H_{\mathrm{occ}}|$ among equal coordinate orders.

Occupation mode

1. maximize $|H_{\mathrm{occ}}|$.

`first` returns one deterministic canonical representative after the primary coordinate optimum and the secondary occupation-stabilizer optimum are proven. It does not claim a globally optimal later physical score. `all` enumerates canonical coordinate ties deterministically up to the configured bound. When all ties are exhausted, they may be ordered by:

1. fewer inequivalent occupied cation orbits under the verified objective stabilizer;
2. canonical occupation or coordinate signature.

When bounded enumeration is truncated, the returned deterministic prefix is not claimed to be the globally best N structures under a secondary ranking. MS4 implements canonical or density ranking only after symmetry is fixed, records whether the tie set was exhaustive, and treats a requested minimum pair distance as a hard feasibility condition. If every retained member fails but the tie set was truncated, it raises an incomplete-filter error rather than claiming global infeasibility.

The existing density score is a species-blind heuristic, not an energy. It must not trade away symmetry. A requested minimum pair distance is a feasibility constraint and must not be hidden as a soft score penalty.

14.3 Bounded tie enumeration

Multiplicity can arise from:

- stabilizer subgroups of equal order that are nonconjugate under the policy-appropriate group C;
- distinct orbit colorings with the same stabilizer;
- distinct exact variant realizations;
- generated structures related by the full parent group.

The proposed API is

```
--max-symmetry-count N
--max-symmetry-types first | all
--max-symmetry-uniqueness occupation | coordinates
```

`first` means return one deterministic representative **after proving the optimal order**. It does not permit an early arbitrary stop.

`all` remains bounded by `--max-symmetry-count`. The report must state when enumeration was truncated. Final emitted structures are deduplicated by their actual occupation or coordinate equivalence under the full parent group; the smaller strict-mode group $G_{\mathcal{V}}$ limits only safe search reduction, not the physical definition of symmetry equivalence between two realized structures.

The current grouped-site canonicalization follows the general symmetry-reduction principle used in derivative-structure and site-occupancy enumeration [3,4,9]. The present stabilizer-maximization objective and policy-dependent conjugacy reduction are narrower package-specific constructions.

14.4 Symmetry-preserving smoothing - implemented in MS5

Independent single-ion relocation is incompatible with a preserved nontrivial subgroup. MS5 therefore acts on complete required-subgroup orbits.

For relocation and species exchange, orbit cardinalities must match and ring-family identity must be preserved. Variant reorientation and target-orbit filling use the MS3 selected-orbit realization test. Missing or ambiguous required transitions are never accepted.

After every trial, MS5 recomputes the actual full-parent coordinate stabilizer. A trial is valid only when it contains the required subgroup. Symmetry gain is allowed; symmetry loss is not.

The current neighborhood deliberately excludes compound repartitioning, such as replacing one large orbit with several smaller orbits. If no improving move in the implemented neighborhood exists, the current symmetry-certified structure is retained and the report states that only a local stopping condition was reached.

15. Module boundaries and public API

15.1 Module boundaries

The new group algorithms should not be added to the existing population monolith.

```
lta_cation_template.py
  immutable template model
  framework and grouped-site geometry
  source-operation provenance

lta_cation_symmetry.py
  affine operations
  complete operation table
  site and framework actions
  symmetry-completed and strict variant libraries
  total or partial variant transitions
  library-preserving group
  action validation and diagnostics

lta_cation_subgroups.py
  subgroup closure and representation
  policy-appropriate conjugacy orbits
  subgroup orbits
  descending-order search

lta_cation_maxsym.py
  occupation constraint resolution and orbit-coloring solver
  exact selected-variant-orbit feasibility
  canonical stabilizers, bounded ties, and optimality certificates

lta_cation_maxsym_io.py
  certified solution assembly
  post-symmetry scoring and hard filters
  reports, writers, and round-trip stabilizer verification

lta_cation_population_grouped.py
  random and unique-batch modes
```

```
backward-compatible CLI and maximum-symmetry dispatcher
shared chemistry, volume, structure assembly, and writers
```

MS0 implements `lta_cation_template.py` and `lta_cation_symmetry.py`; MS1 implements `lta_cation_subgroups.py`; MS2 implements the occupation layer of `lta_cation_maxsym.py`; MS3 implements its exact-coordinate layer; MS4 implements `lta_cation_maxsym_io.py` and the user-facing dispatcher. Optimizer logic remains separate from parsing and serialization. The legacy random path remains unchanged and still uses its historical loader.

15.2 Implemented CLI surface

```
--population-mode random | maximum-symmetry          # default: random
--symmetry-backend declared | spglib | auto          # default: auto
--framework-equivalence chemical                     # default: chemical
--symmetry-realization exact | occupation            # default: exact
--variant-library-policy symmetry-complete | strict  # default: symmetry-complete
--symmetry-tiebreak canonical | density              # default: canonical
--max-symmetry-count N                               # default: 1
--max-symmetry-types first | all                     # default: first
--max-symmetry-uniqueness occupation | coordinates   # default: coordinates
--max-symmetry-search-nodes N                        # default: 1_000_000
--max-coloring-search-nodes N                        # default: 1_000_000
--max-variant-search-nodes N                         # default: 1_000_000
--max-group-order N                                  # default: 4096
--density-sigma ANGSTROM                             # default: 5.0
--min-pair-distance ANGSTROM                         # optional hard filter
--significant-figures N                              # default: 8
```

Topological pseudo-symmetry and idealized central variants are deferred. Symmetry-specific options are active only in `maximum-symmetry` mode. The current random and unique-batch APIs remain valid.

The default `symmetry-complete` policy may add allowed coordinates, but only as exact parent-symmetry images of supplied variants. `strict` preserves the supplied coordinate list exactly and may require testing many more subgroup states.

15.3 Implemented result types

```
SymmetryModel
VariantLibrary
VariantProvenance
SubgroupOrbitPartition
SiteOccupation
OrbitColoringSolution
VariantTransitionTable
VariantAssignment
FeasibilityStatus
OptimalityCertificate
SymmetryScore
MaximumSymmetryBatchResult
MaximumSymmetryMember
PostWriteVerification
MaximumSymmetryWriteResult
```

Every emitted structure reports:

- detected, closure-completed candidate, framework-valid, and site-valid operation counts;
- $|G|$, $|\phi_P(G)|$, $|K_P|$, and $|G_\mathcal{V}|$;
- variant-library policy, input/generated/active counts, and provenance summary;
- variant-transition coverage, missing counts, ambiguity counts, and global-closure status;
- occupation and coordinate stabilizer orders;

- stabilizer generators or operation identifiers;
- number of inequivalent occupied cation orbits under the objective stabilizer;
- tolerance policy and maximum mapping residuals;
- canonical signatures and optional tie-break scores;
- admissible conjugating-group order, subgroup-search nodes, highest exhausted order, indeterminate subgroup counts, and `optimality_proven`;
- whether maximum-order type or structure enumeration was truncated.

16. Staged implementation plan

MS0 - template, parent group, and variant-library hardening - implemented in v7.5

- introduced `LTATemplateModel`;
- retained complete affine operations;
- validated operations against the chemical framework and ring-family site model;
- constructed the physical-site action independently of variant closure;
- implemented deterministic symmetry completion with provenance;
- implemented strict partial variant-transition and residual tables;
- computed and validated $G_{\mathcal{V}}$;
- constructed operation, inverse, and total-action tables;
- added immutable tolerance and validation reports;
- added minimal project/test configuration so the canonical test command works from the package root.

MS0 implementation specifications:

- `doc/specs/lta_cation_template_spec.md` and `.pdf`;
- `doc/specs/lta_cation_symmetry_spec.md` and `.pdf`.

MS1 - subgroup and orbit infrastructure - implemented in v7.6

- represents subgroups by closed parent-operation index sets with deterministic generator witnesses;
- implements subgroup validation, closure, conjugation, and validated object-orbit partitions;
- exhaustively enumerates the subgroup lattice by adjoining parent elements;
- traverses policy-appropriate conjugacy orbits lazily after exhaustive discovery;
- selects $C = G_{\text{site}}$ or $G_{\mathcal{V}}$ from the realization/library policy;
- enforces a stable distinct-subgroup search-node guard and raises `MaximumSymmetrySearchIncompleteError` rather than returning a partial certificate;
- validates C_4, D_4, and the synthetic crystallographic C_4 site action.

MS1 implementation specification:

- `doc/specs/lta_cation_subgroups_spec.md` and `.pdf`.

MS2 - occupation optimizer - implemented in v7.7

- resolves exact counts or historical per-family ratios to integer vacancy/supported-cation constraints;
- implements exact subgroup-orbit coloring with memoized dynamic programming and fast arithmetic rejection;
- proves maximum occupation-stabilizer order using exhaustive G_{site} conjugacy reduction;
- canonicalizes occupations under the full parent action and conjugates construction subgroups consistently;
- recomputes actual stabilizers and verifies the orbit-stabilizer identity;
- supports deterministic `first` and bounded `all` tie modes;
- returns an explicit `OccupationOptimalityCertificate` and raises on incomplete coloring search.

MS2 implementation specification:

- `doc/specs/lta_cation_maxsym_spec.md` and `.pdf`.

MS3 - exact static-coordinate optimizer - implemented in v7.8

- implements completed-library total-action realization;
- implements strict partial-transition selected-orbit closure tests;
- implements the site-stabilizer fixed-representative and all-site equivariance tests;
- implements feasible/infeasible/indeterminate strict feasibility states;
- includes selected-variant realizability in subgroup feasibility;
- uses G_{site} conjugacy reduction for completed mode and $G_{\mathcal{V}}$ reduction for strict mode;
- propagates and combines exact variant assignments under an independent search guard;
- recomputes geometric coordinate stabilizers and full-parent coordinate signatures;
- proves the maximum coordinate-stabilizer order and secondary occupation-stabilizer order with an explicit certificate.

MS3 implementation specification:

- `doc/specs/lta_cation_exact_realization_spec.md` and `.pdf`.

MS4 - integration, final scoring, reports, and serialization - implemented in v7.9

- adds `lta_cation_maxsym_io.py` as the boundary between certified optimization and legacy structure/writer code;
- converts immutable template atoms and active variants into `PopulationResult` without re-reading the CIF;
- supports exact coordinate realization and occupation-only deterministic variant realization;
- recomputes actual occupation and coordinate stabilizers before assembly;
- applies canonical or density ranking only after symmetry is fixed;
- enforces hard minimum pair-distance filters with exhaustive-versus-truncated failure semantics;
- reports variant provenance, transition coverage, reduction policy, search optimality, physical scores, and stabilizer operation IDs;
- exposes `--population-mode maximum-symmetry` through the existing CLI while keeping random mode as the default;
- serializes one or bounded multiple structures through the existing CIF, POSCAR, and LAMMPS writers;
- reads every verified output back, matches periodic species-labeled point sets, recomputes the full parent-operation stabilizer, and requires exact stabilizer equality;
- removes files created by a failed verified batch.

MS4 implementation specification:

- `doc/specs/lta_cation_maxsym_io_spec.md` and `.pdf`.

MS5 - symmetry-coupled smoothing - implemented in v7.10

- adds `lta_cation_symmetry_smoothing.py`;
- validates a default or user-supplied required coordinate subgroup;
- partitions physical sites into required-subgroup orbits;
- adds complete-orbit variant reorientation, occupied-to-empty relocation, and equal-size species exchange;
- uses the existing density score, soft pair target, and optional hard pair constraint;
- propagates target variants through exact selected-orbit realizations;
- rebuilds every trial through the MS4 integration boundary;
- retains and geometrically reverifies the required stabilizer after every accepted move;
- adds deterministic steepest descent, candidate/variant search guards, batch uniqueness checks, reports, and smooth-CLI integration;
- retains mandatory CIF/POSCAR/LAMMPS read-back stabilizer verification.

MS5 implementation specification:

- `doc/specs/lta_cation_symmetry_smoothing_spec.md` and `.pdf`.

MS6 - exact-search acceleration and scale validation - implemented in v7.11

- detects a unique parent-equivariant active variant at every physical site;

- proves $H_{\text{coord}} = H_{\text{occ}}$ for that domain and lifts the MS2 optimum directly;
- builds one species-independent exact-realization catalog per tested subgroup;
- reuses catalog entries across occupation colorings without consuming additional variant-search nodes;
- extends coordinate certificates with search-strategy and cache diagnostics;
- extends reports with subgroup conjugacy keys, generators, finite operation orders, affine descriptors, and occupied orbit summaries;
- adds the 46-site synthetic LTA-capacity fixture with 24 four-ring, 16 six-ring, and 6 eight-ring sites;
- certifies representative order-2 and identity cases with `max_variant_search_nodes=1`;
- retains mandatory read-back stabilizer equality.

MS6 implementation specification:

- `doc/specs/lta_cation_exact_search_acceleration_spec.md` and `.pdf`.

MS7 - symmetry construction and search acceleration - implemented in v7.12

MS7 removes measured cold-path bottlenecks without changing the exact objective or output certification.

- Framework actions use expansion provenance algebra, $h(a, g) = (a, hg)$, with direct residual validation and a vectorized geometric fallback.
- Symmetry-completed variants are constructed by breadth-first closure under a deterministic finite generating set, then validated under every parent operation.
- Subgroup states use integer bit masks internally while retaining the public immutable `Subgroup` API.
- Small raw occupation spaces use exact direct enumeration when

$$N_{\text{raw}} = \prod_r \frac{N_r!}{\prod_\alpha N_{r,\alpha}!}$$

  does not exceed the configured limit.
- MS5 scores an existing density report rather than recomputing the same periodic distances.
- Strategy diagnostics are included in symmetry and optimizer certificates.

For the real primitive Na-LTA template, MS7 retains parent order 48 and the certified order-four Na-LTA stabilizer. The benchmark contract and exact measurements are recorded in `MS7_BENCHMARK_RESULTS.txt` and `validation/ms7_benchmark_results.json`.

MS7 implementation specification:

- `doc/specs/lta_cation_performance_optimization_spec.md` and `.pdf`.

17. Verification strategy

The implementation must include the following categories.

17.1 Algebraic group tests

1. identity, closure, inverse, and multiplication-table consistency;
2. physical-site and framework total-action composition;
3. selected-orbit variant-transition composition and inverse consistency;
4. explicit action convention
$$p_g[i] = j \iff g(s_i) = s_j;$$
5. cases where distinct affine operations share one physical-site permutation;
6. kernel and image order checks;
7. known cyclic and dihedral subgroup lattices.

17.2 Orbit and optimizer tests

1. exact population-feasibility and infeasibility fixtures;
2. full-parent conjugacy preserves occupation feasibility;

3. full-parent conjugacy preserves exact feasibility for a symmetry-completed library;
4. a strict incomplete library can break exact feasibility equivalence under full-parent conjugacy;
5. strict mode still finds the optimum by searching all required subgroups or $G_{\mathcal{V}}$-conjugacy orbits;
6. feasibility monotonicity down the subgroup lattice;
7. equal-order nonconjugate optimum types;
8. uniform occupation has stabilizer G;
9. one occupied site has its site stabilizer;
10. complement colorings have the expected stabilizer;
11. exact-mode fixtures with $H_{\mathrm{occ}} > H_{\mathrm{coord}}$;
12. exact mode never falls back silently to occupation mode;
13. brute-force or MILP comparison for small systems.

For every small fixture, verify the orbit-stabilizer identity

$$|\operatorname{Orb}_G(c)|\,|\operatorname{Stab}_G(c)| = |G|.$$

17.3 Geometry and determinism tests

1. physical-site mapping is unchanged when unused variants are removed;
2. symmetry completion generates only parent-symmetry images and records provenance;
3. the completed library carries a unique composition-consistent total action;
4. $G_{\mathcal{V}}$ is a validated subgroup and equals G for a globally closed strict library;
5. an incomplete strict library can still realize a closed selected orbit;
6. a missing strict transition required by a selected orbit makes that representative infeasible;
7. an ambiguous required strict transition propagates indeterminate status and prevents an exact optimality certificate;
8. selected-orbit transitions obey the group table across periodic boundaries;
9. deterministic result independent of CIF atom-row order;
10. deterministic site and generated-variant ordering independent of connected-component discovery order;
11. operation-count, variant-provenance, and transition-coverage diagnostics are correct;
12. tolerance-sensitivity fixtures near acceptance and periodic-deduplication boundaries.

17.4 End-to-end tests

1. post-write round-trip verification for CIF, POSCAR, and LAMMPS data;
2. output rounding does not lower the reported stabilizer;
3. bounded `all` mode reports truncation;
4. `first` mode proves the optimum before returning;
5. the search-node limit raises an incomplete-search error and emits no maximum claim;
6. unresolved higher-order strict-mode ambiguity raises an ambiguity error and emits no maximum claim;
7. completed exact mode is invariant under full-parent subgroup conjugacy;
8. strict exact mode succeeds when unused variants are incomplete but selected orbits are closed;
9. strict mode does not skip a feasible full-parent conjugate outside the same $G_{\mathcal{V}}$ orbit;
10. regression of all current random and unique-batch behavior;
11. symmetry-coupled smoothing never lowers the required stabilizer;
12. complete-orbit relocation and exchange preserve per-ring species counts;
13. equivariant variant reorientation preserves occupation;
14. candidate and variant search limits emit no false local-optimum claim;
15. the smooth maximum-symmetry CLI passes post-write stabilizer verification;
16. forced-variant LTA-capacity order-2 and order-1 cases succeed under a one-node variant budget;
17. forced-path coordinate and occupation stabilizers are identical after geometric recomputation;
18. subgroup catalogs are reused without additional representative-test nodes;
19. multi-variant search reports the catalogued strategy and cache accounting;
20. LTA-capacity low-symmetry CIF output round-trips with the certified stabilizer.
21. provenance-derived and vectorized fallback framework actions satisfy the same action contract;
22. generator-orbit variant completion is closed under every parent operation;

23. direct and subgroup-orbit occupation solvers agree on small exhaustive fixtures;
24. bit-mask subgroup closure reproduces known subgroup lattices;
25. the real 48-operation Na-LTA regression retains coordinate order four.

18. Current limitations and deferred scope after MS7

- The minimal CIF parser is intentionally incomplete and does not support every CIF syntax feature.
- The legacy random unique-batching path still uses declared CIF operations, periodic grouped-site centroids, and the grouped-site permutation-image order; a P1 input therefore gives identity-only legacy deduplication.
- The MS0-MS7 maximum-symmetry path retains complete affine operations, validates framework/site actions, supports declared/spglib/auto backends, treats off-center variants explicitly, accelerates forced and repeated orbit realizations, preserves a required subgroup during local density repair, and verifies serialized outputs.
- Legacy unique batching ignores off-center variants by design.
- Legacy random single-ion smoothing can lower nontrivial coordinate symmetry and remains isolated from the maximum-symmetry path.
- MS5 preserves a required subgroup but is a local search; it does not prove a globally optimal density score.
- The implemented MS5 neighborhood uses equal-size complete-orbit relocation and exchange. Compound multi-orbit repartitioning is deferred.
- Exact mode is relative to the chosen active variant-library policy and declared tolerances.
- MS6 removes the observed low-symmetry variant bottleneck when the active coordinate choice is uniquely forced. MS7 removes the measured framework-action, variant-completion, and subgroup-representation bottlenecks. General multi-variant optimum-order tie enumeration can still be combinatorial and remains protected by explicit coloring and variant-search guards.
- The default symmetry-completed policy expands the allowed coordinate library only by exact parent-symmetry images of supplied variants; it does not infer energetic stability of those images.
- Strict policy preserves the supplied coordinate list exactly. Its exact optimum can differ among full-parent conjugate subgroups, so search reduction is limited to $G_{\mathcal{V}}$.
- Cell-enlarging orderings are outside the planned fixed-cell feature.
- Topological Si/Al pseudo-symmetry is deferred.
- Idealized central cation positions are deferred and must never be introduced silently.
- The volume model is an affine approximation for the Al24Si24O96 scope and neglects excess mixing volume, temperature variation among references, hydration differences, and local ordering. Non-24-Al cells and other Si:Al ratios are deferred.
- Random populations and the density heuristic are not thermodynamic sampling.

19. Algorithmic provenance and references

19.1 Provenance boundary

The package combines standard external methods with project-specific design:

- Disjoint-set forests for connected-component grouping follow Tarjan's set-union framework [6].
- Symmetry reduction of occupational configurations follows the general finite group-action approach used by Hart and Forcade, Grau-Crespo *et al.*, and Lian *et al.* [3,4,9].
- Permutation representation, finite generating sets, subgroup closure, conjugacy, orbit, and stabilizer language follow standard computational group theory; Seress is the principal algorithmic reference [7].
- Group-subgroup orbit splitting is motivated by WYCKSPLIT [5], while the grouped LTA sites are custom project objects.
- Optional numerical symmetry detection and equivalent-atom information use spglib [8,12].
- The provenance-derived framework action, deterministic generator-orbit variant completion, bit-mask subgroup representation, hybrid direct/subgroup occupation solver, symmetry-completed and strict variant-library policies, library-preserving group $G_{\mathcal{V}}$, policy-dependent conjugacy reduction, partial variant-transition model, selected-orbit closure criterion, partial-transition site-stabilizer theorem, joint occupation/variant feasibility formulation, lexicographic maximum-symmetry ranking, forced-variant

theorem, subgroup-orbit realization catalog, explicit optimality certificate, and orbit-coupled smoothing policy are derived specifically for this package from standard group-action principles.

- The alkali LTA dry-volume data come from Carey *et al.* and Carey's thesis [1,2]. Mg, Ca, Sr, and Ba bindings come respectively from Adams and Rees, Porcher *et al.*, Pluth and Smith, and Kim *et al.* [14-17]. The Mg source retains residual water; Sr is a questionable effective corner rather than a clean ideal endmember; Ba is an affine extrapolation from a dry partially exchanged composition.
- Structure representation and CIF/VASP/LAMMPS serialization are delegated to pymatgen [10], with LAMMPS data semantics cross-checked against the LAMMPS documentation and canonical software reference [11]. NumPy supplies the array-programming substrate used throughout the numerical implementation [18].
- The species-labeled post-write point-set comparison uses the standard augmenting-path characterization of matchings due to Berge [13]; the periodic matching tolerance, stabilizer-equality certificate, and failure policy are package-specific.
- The ion-centered density score and greedy relocation policy are original heuristics and are not claimed as published algorithms.

19.2 References

1. T. Carey, C. C. Tang, J. A. Hriljac, and P. A. Anderson, "Chemical Control of Thermal Expansion in Cation-Exchanged Zeolite A," *Chemistry of Materials* **26**, 1561-1566 (2014). DOI: 10.1021/cm403312q.

2. T. Carey, *Chemical control of thermal expansion in zeolites with the LTA topology*, PhD thesis, University of Birmingham (2013), Chapter 3. Stable repository record: https://etheses.bham.ac.uk/id/eprint/4258/.

3. G. L. W. Hart and R. W. Forcade, "Algorithm for generating derivative structures," *Physical Review B* **77**, 224115 (2008). DOI: 10.1103/PhysRevB.77.224115.

4. R. Grau-Crespo, S. Hamad, C. R. A. Catlow, and N. H. de Leeuw, "Symmetry-adapted configurational modelling of fractional site occupancy in solids," *Journal of Physics: Condensed Matter* **19**, 256201 (2007). DOI: 10.1088/0953-8984/19/25/256201.

5. E. Kroumova, J. M. Perez-Mato, and M. I. Aroyo, "WYCKSPLIT: a computer program for determination of the relations of Wyckoff positions for a group-subgroup pair," *Journal of Applied Crystallography* **31**, 646 (1998). DOI: 10.1107/S002188989800712X.

6. R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," *Journal of the ACM* **22**, 215-225 (1975). DOI: 10.1145/321879.321884.

7. A. Seress, *Permutation Group Algorithms*, Cambridge Tracts in Mathematics 152, Cambridge University Press (2003).

8. A. Togo, K. Shinohara, and I. Tanaka, "Spglib: a software library for crystal symmetry search," *Science and Technology of Advanced Materials: Methods* **4**, 2384822-2384836 (2024). DOI: 10.1080/27660400.2024.2384822.

9. J.-C. Lian, H.-Y. Wu, W.-Q. Huang, W. Hu, and G.-F. Huang, "Algorithm for generating irreducible site-occupancy configurations," *Physical Review B* **102**, 134209 (2020). DOI: 10.1103/PhysRevB.102.134209.

10. S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, and G. Ceder, "Python Materials Genomics (pymatgen): A robust, open-source Python library for materials analysis," *Computational Materials Science* **68**, 314-319 (2013). DOI: 10.1016/j.commatsci.2012.10.028. Current API documentation: https://pymatgen.org/.

11. A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton, "LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," *Computer Physics Communications* **271**, 108171 (2022). DOI: 10.1016/j.cpc.2021.108171. Command documentation: https://docs.lammps.org/.

12. Spglib documentation, "Spglib dataset" and "Definitions and conventions," describing returned symmetry operations, equivalent atoms, and tolerance-dependent symmetry search: https://spglib.readthedocs.io/.

13. C. Berge, "Two Theorems in Graph Theory," *Proceedings of the National Academy of Sciences of the United States of America* **43**, 842-844 (1957). DOI: 10.1073/pnas.43.9.842.

14. J. M. Adams and L. V. C. Rees, "The structure of a dehydrated partially magnesium exchanged zeolite A (~Mg2Na8-A) by neutron profile refinement," *Journal of Solid State Chemistry* **62**, 184-190 (1986). DOI: 10.1016/0022-4596(86)90230-6.

15. F. Porcher, M. Souhassou, H. Graafsma, A. Puig-Molina, Y. Dusausoy, and C. Lecomte, "Refinement of framework disorder in dehydrated CaA zeolite from single-crystal synchrotron data," *Acta Crystallographica B* **56**, 766-772 (2000). DOI: 10.1107/S0108768100006893.

16. J. J. Pluth and J. V. Smith, "Crystal structure of dehydrated strontium-exchanged zeolite A. Absence of near-zero-coordinate strontium(2+) ion. Presence of aluminum complex," *Journal of the American Chemical Society* **104**, 6977-6982 (1982). DOI: 10.1021/ja00389a015.

17. Y. Kim, V. Subramanian, R. L. Firor, and K. Seff, "Four Crystal Structures of Ba_xNa_(12-2x)-A (1 <= x <= 6) Relating to the Instability of Barium-Exchanged Zeolite A Toward Dehydration," *ACS Symposium Series* **135**, 137-153 (1980). DOI: 10.1021/bk-1980-0135.ch007.

18. C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernandez del Rio, M. Wiebe, P. Peterson, P. Gerard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature* **585**, 357-362 (2020). DOI: 10.1038/s41586-020-2649-2.

v7.15 architecture delta: sparse chloride occlusion

The v7.15 random path adds a provenance layer above the existing grouped-site machinery rather than a second placement architecture. Framework and excess cation roles share one physical-site inventory; framework occupation is realized first with the legacy RNG and excess occupation is realized second with an isolated deterministic stream.

The final physical structure has one authoritative assembly owner: framework atoms + all grouped-site cations + validated chloride cage guests. Cage selection is downstream of random cation smoothing. The density subsystem therefore remains cation-only.

Occlusion setting validation is fail-closed to the canonical `tests/real_lta_primitive.cif` identity and the expected alpha/beta cage-center orbits. Arbitrary origin/setting remapping is intentionally absent.

MS0-MS7 and the symmetry-unique random signature remain cation-state models. v7.15 consequently rejects nonzero occlusion before maximum-symmetry optimization/certification and before multi-member unique batching. Extending the group action to cage states is a future architecture change, not an implicit property of v7.15.